

Disaster Damage Assessment via Vision-Language Model Fine-Tuning

Project brief

Build a proof-of-concept that fine-tunes an open-weight vision-language model to assess structural damage in post-disaster satellite imagery, conditioned on stated event context. The deliverable is a working pipeline, a measured base-vs-tuned comparison, and a demo interface.

The primary goal is hands-on fluency with VLM adaptation. The secondary goal is a portfolio artifact with an operational hook for disaster response work.

Why a VLM rather than a classifier

xBD has strong CNN baselines. This project is not trying to beat them on accuracy. It is demonstrating a capability a classifier structurally cannot provide:

1. **Context conditioning.** The same collapsed roof means different things after a hurricane than after a wildfire. A classifier cannot take “event type” as an input and change its reasoning. A VLM can.
2. **Natural language output with cited evidence.** The model states what visual evidence drove the damage grade, which is what makes an assessment reviewable by a human adjuster or responder.
3. **Zero-shot floor.** A pretrained VLM already produces a plausible answer. That gives a real baseline to measure improvement against, which a randomly initialized classifier does not.

Frame the writeup around this. The headline number is the base-vs-tuned delta, not the absolute accuracy.

Dataset

xBD (xView2 Challenge dataset)

- Pre- and post-event high-resolution satellite image pairs
- Roughly 850k building polygons across 19 disaster events
- Event types span hurricane, wildfire, flood, tsunami, earthquake, volcanic eruption
- Damage labels use a four-level ordinal scale: no damage, minor damage, major damage, destroyed
- The scale derives from an operational joint damage assessment standard, not an invented rubric

Download from the xView2 site. Verify the license terms and record them in the repo.

Practical notes

- The imagery is large. Plan disk before download.
 - Building polygons are supplied as GeoJSON with damage attributes on the post-event annotations.
 - Class distribution is heavily skewed toward “no damage.” This must be handled explicitly.
 - The official train/test split exists. Use it. Do not invent your own.
-

Task design

Core task

Given a post-event image chip and a context prompt, output a structured damage assessment.

Prompt contains: event type, days since event, structure type where known.

Output contains: damage grade on the four-level scale, the visual evidence supporting that grade, and a triage priority call.

Chipping strategy

Full satellite tiles are too large and contain too many buildings for a single VLM judgment. Crop around individual building polygons with surrounding context, so each training example is one building with enough neighborhood visible to reason about.

Decide and document: chip size, context padding, and whether to include the pre-event chip alongside the post-event chip. Starting simple with post-event only is defensible; the

pre/post pair is a strong second experiment.

Output schema

Fix the schema before generating any data. Every training example must use identical field names, identical field order, identical phrasing conventions. The model learns format alongside content, and inconsistency wastes adapter capacity.

Keep the schema parseable. It needs to be machine-readable for evaluation.

Class imbalance

This is the most likely cause of a disappointing result. Address it up front.

The “no damage” class dominates. A model that always answers “no damage” will score well on naive accuracy and be useless. Options to consider and document:

- Stratified sampling to a balanced or near-balanced training set
- Deliberate oversampling of minor and major damage, which are the hardest and most operationally important classes
- Reporting per-class metrics rather than only overall accuracy

The minor/major boundary is where human annotators disagree most. Expect it to be where the model struggles, and say so in the writeup rather than hiding it.

Model and tuning approach

Base model: Qwen2.5-VL-7B-Instruct. Start here. Only move to a larger model if 7B plateaus above the target error.

Method: LoRA, optionally with 4-bit quantization of the frozen base weights.

What is frozen: the vision encoder. It already knows what rubble and burn scars look like as texture. The adaptation needed is in the language model, which is where the mapping from visual evidence to damage grade and output format lives.

What is trained: low-rank adapters on the attention and MLP projections of the language model. Consider whether to also train the projector that maps vision embeddings into token space; freezing it is the simpler default.

Configuration starting points: rank in the 16 to 32 range, alpha at twice the rank, learning

rate an order of magnitude or more above full fine-tuning, two to three epochs. Cosine schedule with warmup. Watch validation loss and stop when it turns.

Framework: LLaMA-Factory for configuration-driven simplicity, or Unsloth for speed and memory headroom on a single card. Either handles Qwen2.5-VL. Pick one and stay with it.

Pipeline stages

1. **Acquire.** Download xBD. Verify integrity. Record license and version.
 2. **Parse.** Read the GeoJSON annotations into a flat tabular label file. One row per building: image id, event id, event type, polygon, damage grade, chip path once generated.
 3. **Chip.** Generate per-building image crops from the label file. Deterministic and regenerable.
 4. **Build conversations.** Script that reads the label file and emits chat-format training data. Templated. Never hand-edited.
 5. **Split.** Group by event, not by building or by tile. Buildings from one disaster event must not appear in both train and test. Random splitting leaks badly here and inflates results.
 6. **Train.** Configuration file pointed at train and validation sets.
 7. **Evaluate.** Run the held-out test set through both the base model and the tuned adapter. Parse outputs. Emit a metrics table plus per-example predictions for inspection.
 8. **Serve.** Simple interface for the demo.
-

Repository structure

```
data/  
  raw/                xBD as downloaded, never modified  
  labels.csv          parsed annotations, the source of truth  
  chips/              generated crops, gitignored  
  processed/          generated training jsonl, gitignored  
scripts/  
  parse_annotations  
  build_chips  
  build_dataset  
  split  
  evaluate  
configs/
```

```
model and training configuration
outputs/
  adapters/
  eval/
app
```

Evaluation

Primary metric: ordinal accuracy on the four-level damage scale, reported per class and overall.

Secondary metrics:

- Adjacent-class error rate. Confusing minor with major is a different failure than confusing no-damage with destroyed. Report them separately.
- Mean absolute error treating the scale as ordinal, which penalizes distant confusions appropriately.
- Unparseable output rate. Count outputs that do not conform to the schema. This is a real metric and belongs in the results table.

Mandatory baseline: the base model, zero-shot, on the identical test set with the identical prompt. Without this number the project has no argument.

Discipline: the validation set is for deciding when to stop training. The test set is scored once, at the end, for reporting. Tuning against test numbers between runs converts test into validation and inflates the headline.

The context-conditioning demonstration

This is the centerpiece of the demo and should be built deliberately.

Hold the image constant. Change only the stated event type in the prompt. Show the assessment change. Wildfire damage and flood damage present differently, and a model that has learned the conditioning will reason differently about identical pixels.

Second variant: hold image and event type constant, change days since event. Recency changes what is expected to be visible and what triage priority is appropriate.

If the tuned model shows this behavior and the base model does not, that contrast is the strongest single result in the project.

Known failure modes to investigate and report

These are expected. Finding them and characterizing them honestly is more valuable than a clean number.

- Overcalling destroyed where smoke, shadow, or cloud obscures a structure
- Undercalling major damage where the damage is on a facade not visible from nadir view
- Poor discrimination at the minor/major boundary, mirroring human annotator disagreement
- Degradation on event types underrepresented in training
- Format drift on unusual inputs, producing unparseable output

Reproducibility requirements

Write a run configuration file into every adapter output directory containing the exact prompt template string, base model revision, all hyperparameters, and a hash of the training dataset.

An adapter is meaningless without the prompt template it was trained on. Inference must reconstruct the prompt identically or performance degrades with no visible cause.

Scope and sequencing

Phase 1. Data acquisition, parsing, chipping, split. This is the largest time investment. Do not start training until the label file and split are correct.

Phase 2. Evaluation harness with base model baseline. Build this before the training config. Knowing what good looks like before training starts prevents wasted runs.

Phase 3. First training run. Expect it to be wrong. Iterate on the prompt template and class balance.

Phase 4. Context-conditioning demo and writeup.

Phases 1 and 2 are the majority of the work. Phase 3 is hours of compute.

Out of scope

- Beating published xView2 leaderboard scores
 - Full-tile inference or a production serving path
 - Multi-model comparison
 - Any modeling of economic loss or insurance value
-

Definition of done

- A tuned adapter that outperforms the zero-shot base model on the held-out test set, with the delta reported per class
- An evaluation script that reproduces the headline numbers from a single command
- A demo showing the context-conditioning behavior on a fixed image
- A writeup stating the method, the numbers, the failure modes found, and the honest limitations